APPENDIX

A. Comparison with Active Object Tracking Methods

Active Object Tracking (AOT) methods, as described in [29], utilize an end-to-end approach through reinforcement learning. These methods process raw video frames as input and generate camera movement actions as output. According to [29], there are seven discrete actions: *move-forward/backward, turn-left/right, move-forward-and-turn-left/right*, and *no-op*. However, existing person-following datasets, such as Chen's dataset [5] and our own dataset, only provide ground truth bounding boxes of the target person. For a valid comparison, we need to map these bounding boxes to the action space. As noted in [29], a two-stage method (combining single object tracking with a PID controller) can achieve a 100% success rate if the estimated target bounding box (bbox) is accurate. Therefore, we can deduce the ground truth actions from the ground truth target bboxes. This approach allows us to evaluate AOT methods within person-following datasets. According to [29], the objective of the camera action is to keep the target person's bbox centered in the image, maintaining the same size as initially observed. The ground truth action is generated according to the horizontal error $X_{err} = \frac{X_b - W/2}{W/2}$ and the size error $S_{err} = \frac{W_b \times H_b - W_{exp} \times H_{exp}}{W_{exp} \times H_{exp}}$ shown as in Fig. 5. According to [29], we have the following ground truth action mappings:

(1) *Move forward* if $\mathrm{abs}(X_{err}) \leq 0.1$ and $S_{err} \leq -0.2$;
(2) *Move backward* if $\mathrm{abs}(X_{err}) \leq 0.1$ and $S_{err} \geqslant 0.2$;
(3) *No-op* if $\mathrm{abs}(X_{err}) < 0.1$ and $\mathrm{abs}(S_{err}) < 0.2$;
(4) *Move forward and turn right* if $0.1 \leq X_{err} \leq 0.3$;
(5) *Turn right* if $X_{err} > 0.3$;
(6) *Move forward and turn left* if $-0.3 \leq X_{err} \leq -0.1$;
(7) *Turn left* if $X_{err} < -0.3$.

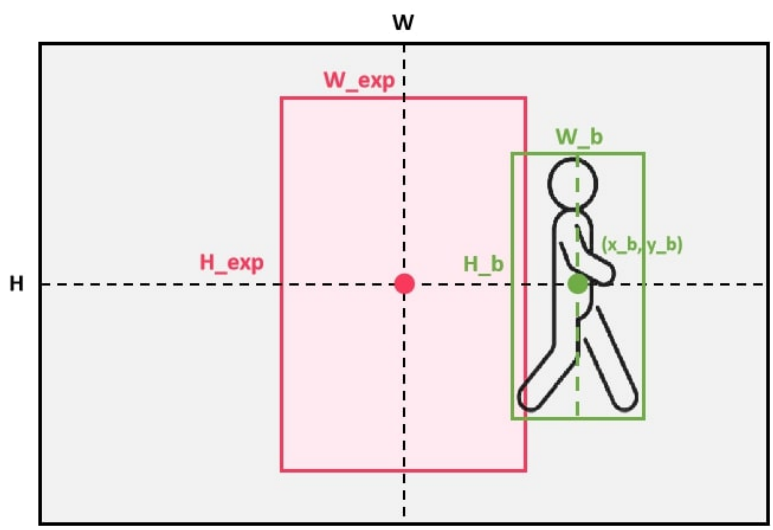
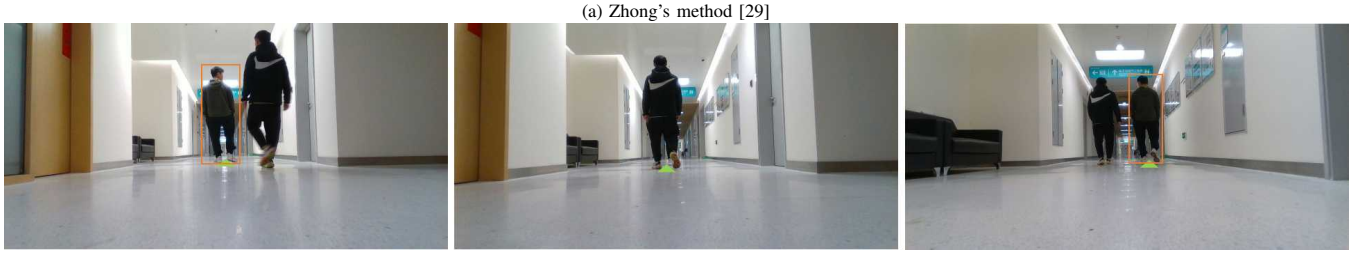


Fig. 5. An example to illustrate errors is as follows: The goal of a correct action is to move the target person closer to the center of the image. Here, (W, H) represents the image's width and height. (W_{exp}, H_{exp}) denotes the size of the expected centered bounding box, initialized by the first bounding box of the target person. (W_b, H_b) represents the target person's bounding box size in the current frame, and (x_b, y_b) is its center point.

This paper aims to evaluate the algorithms' tracking performance, meaning true target person identification is the first priority. Therefore, besides evaluating accurate action estimation, we reduce the matching standards. Specifically, we consider an action to be true if this action tries to move the target person to the center of the image. For example, if the bbox of the target person is on the left of the image, *move backward, turn left*, and *move forward and turn left* are all considered as true actions. The results are reported in Table. I. An example is shown in Fig. 6. We observe that after a long occlusion by a visually similar person, Zhong's method outputs *move forward and turn left*, failing to re-identify the target person. In contrast, our method reliably re-identifies the target person even after long-term occlusion.

(a) Zhong's method [29]



(b) Ours

Fig. 6. An example comparing Zhong's method [29] and ours. From left to right, the sequence represents observations where a long-term occlusion occurs. (a) Zhong's method outputs *move forward and turn left* due to a failed re-identification of the target person. (b) Our method reliably re-identifies the target person even after long-term occlusion.

B. Memory Examples

Several replayed examples of our short-term and long-term memories are shown in Fig. 5. For more visual examples of short-term and long-term memories, please refer to the supplementary video.

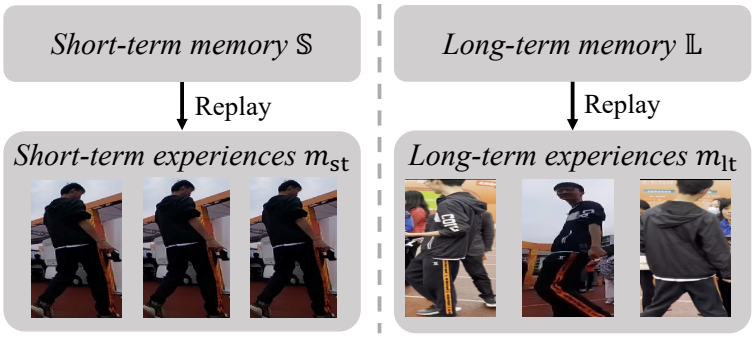


Fig. 7. Both short-term and long-term experiences $(m_{\mathrm{st}} \cup m_{\mathrm{lt}})$ are responsible for training the feature extractor with m_{st} and m_{lt} being sampled from short-term memory $\mathbb{S}$ and long-term memory $\mathbb{L}$, respectively. $\mathbb{L}$ contains sparse yet valuable historical samples, maintained by the *memory manager*. Besides, the target classifier is trained with m_{st} sampled from $\mathbb{S}$, which stores the most recently observed samples, representing the latest knowledge.

C. Target-ReID Lifecycle

Algorithm 1: Target-ReID Lifecycle

Input: Current image $\mathbf{I}$ and tracked people $\{\mathbf{B}, \mathbf{p}\}_i$ representing bounding boxes and positions, target person's identity id, target confidence s, short-term memory $\mathbb{S}$, long-term memory $\mathbb{L}$, feature extractor f and target classifier g

Output: Target person's position $\{\mathbf{p}\}_{id}$ in the current frame

```
Extract image patches M from I and B;
Construct the observation set {M, y}_i where y = 1 if i == id, otherwise y = 0;
Extract features F from M with f;
if id ∈ {i} then
    Estimate s of the target person based on Eq. 3;
    if s > δ_sw then
        Consider {ī} as identities of negative tracks;
        {M, y}_id → S, {M, y}_ī → S based on FILO rule;
        Sample m_st from S;
        Train g with m_st based on Eq. 4;
        ### Separate Thread ###
        {M, y}_ī → L based on FILO rule;
        {M, y}_id → L if it is a keyframe based on Eq. 5;
        Consolidate L with OCL techniques if L is full;
        Sample m_lt from L;
        Train f with m_st and m_lt based on Eq. 1;
        ### Separate Thread ###
        Return target position {p}_id;
    else
        Let id = −1, indicates id switch between the target person and other people;
        Return ø;
else
    Estimate s of the i_th person based on Eq. 3;
    if s > δ_reid for consecutive ζ_reid frames then
        Let id = i, indicates successful target person ReID;
        Return target person's position {p}_id;
    else
        Return ø;
```